

Jev-Mem: System-One-Controlled Agentic Memory for Efficient AI Agents

Dongming Jiang, Yi Li, Bingzhe Li*

Department of Computer Science, The University of Texas at Dallas
{dongming.jiang, yi.li3, bingzhe.li}@utdallas.edu

Abstract

Agentic memory is becoming essential for long-horizon AI agents, yet many existing systems rely on autoregressive LLMs to control how memories are organized, retrieved, and used, placing expensive generation on the critical path of memory operations. We introduce **Jev-Mem**, a new agentic memory architecture inspired by System-One/System-Two cognition. System One captures fast, lightweight decision-making, whereas System Two performs slower, deliberative reasoning. Jev-Mem brings this division of labor to agentic memory through a dedicated System-One control plane, a structured multi-relational memory plane, and a System-Two reasoning plane. The System-One controller governs memory typing and relational organization during construction, and dynamically performs query routing, retrieval-budget allocation, graph traversal, candidate scoring, and adaptive stopping during retrieval. System Two is invoked only for complex reasoning and answer synthesis. This design improves both memory effectiveness and system efficiency: on LoCoMo Jev-Mem achieves an overall LLM-as-a-Judge score of 0.777, an 11.0% relative improvement over the strongest baseline, while reducing memory construction time to 158 s, a $6.6\times$ speedup over the fastest competing memory system, and lowering average query latency to 0.93 s, a 36.7% reduction. The code of Jev-Mem is publicly available.¹

1 Introduction

Large language model (LLM) agents are increasingly expected to operate as persistent systems over long interaction horizons, supporting applications such as coding assistants, personal agents, research agents, and autonomous workflows (Brown et al., 2020; Achiam et al., 2023; Wei et al., 2022). Such agents typically interleave reasoning with tool invocation and environment actions (Yao et al., 2023b;

Schick et al., 2023), and operate over multi-step environments such as interactive web platforms and software repositories (Zhou et al., 2024; Yang et al., 2024). During these interactions, agents continuously accumulate user preferences, task history, and environment knowledge, quickly exceeding what can be maintained within a fixed context window (Beltagy et al., 2020; Liu et al., 2024; Press et al., 2021). Enlarging the nominal context length does not by itself resolve this problem, since models do not reliably exploit all positions of a long input (Hsieh et al., 2024), and long-term interactive settings introduce additional indexing, retrieval, and reading challenges (Lee et al., 2024; Wu et al., 2024; Hu et al., 2026). To remain effective over time, agents therefore need mechanisms that can retain useful experience beyond the prompt and recover it when needed. This requirement has made *agentic memory* which has the ability to preserve, organize, update, and retrieve past experience (Xu et al., 2025; Nan et al., 2025; Chhikara et al., 2025; Jiang et al., 2026a; Liu et al., 2026; Jiang et al., 2026c).

Recent work has transformed agentic memory from passive storage into an active and structured component of LLM agents. Early systems mainly store past interactions and retrieve them through semantic similarity, while newer approaches selectively retain salient information, consolidate repeated observations, and organize memory across hierarchical stores. Procedural memory further captures reusable skills, distilled experience, and recurring workflows (Wang et al., 2023; Zhao et al., 2024; Wang et al., 2025). More recent relational approaches represent memories through knowledge graphs or multiple graph views, enabling retrieval over semantic, temporal, causal, and entity relationships rather than isolated memory fragments. As memory becomes richer, however, controlling it becomes increasingly expensive. Persistent agents must repeatedly decide what to store, update, con-

*Corresponding author

¹<https://github.com/libingzheren/Jev-Mem>

nect, retrieve, and when to stop searching. Existing systems typically rely on either fixed heuristics or general-purpose autoregressive LLMs: the former are efficient but inflexible, while the latter provide semantic flexibility at the cost of repeated token generation. As these decisions appear throughout memory construction and retrieval, memory control itself can become a major source of latency and inference overhead.

This observation leads us to reconsider agentic memory through the lens of *System One* and *System Two*, a distinction drawn from dual-process accounts of human reasoning that separate fast, automatic processing from slower and more deliberative thought (Evans, 2008). Many memory operations including memory typing, relation judgment, query routing, candidate scoring, and stopping are structured, high-frequency decisions that naturally fit lightweight System-One computation, while evidence synthesis and final answer generation remain better suited to System Two. Jev (TypeSafe AI, 2026) makes such a separation practical by producing typed probabilistic decisions without autoregressive generation. Yet several questions remain: *How should System One be integrated across the memory lifecycle? How can a weaker reasoner control memory without hurting downstream accuracy? What memory structure and retrieval process best support lightweight control?*

Inspired by this observation, we propose **Jev-Mem**, a new System-One/System-Two architecture that treats memory control itself as a first-class systems layer. Unlike existing agentic memory systems that rely on heuristics or repeatedly invoke autoregressive LLMs throughout memory construction and retrieval, Jev-Mem introduces a dedicated System-One control plane that handles high-frequency structured decisions, a shared structured memory data plane, and a System-Two reasoning plane reserved for complex synthesis and answer generation. This separation redesigns the full memory lifecycle: on the write path, System One performs memory typing, redundancy filtering, and semantic, temporal, causal, and entity relation construction; on the read path, it performs query routing, retrieval-budget allocation, graph traversal, candidate scoring, evidence assessment, and adaptive stopping. By moving these frequent decisions out of the generative reasoning loop, Jev-Mem reduces unnecessary autoregressive inference while still preserving strong reasoning capability through System Two. Jev provides one concrete realization

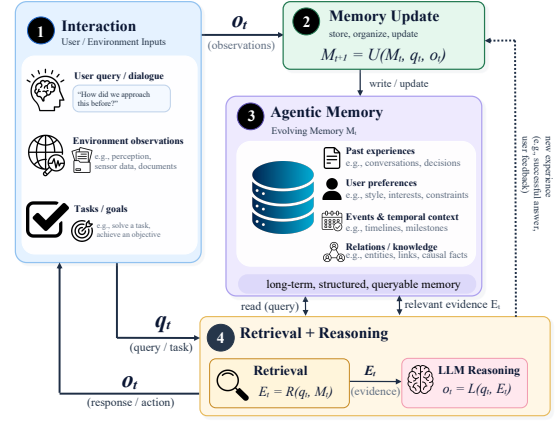


Figure 1: Workflow of agentic memory.

of the System-One controller through typed probabilistic decisions, but the key novelty of Jev-Mem is the architectural separation of lightweight memory control from deliberative reasoning across both memory construction and retrieval.

We make four main contributions.

- We identify an opportunity to leverage System-One-style lightweight decision-making to enable more efficient agentic memory.
- We introduce **Jev-Mem**, a System-One/System-Two architecture that separates lightweight structured memory control from expensive generative reasoning.
- We develop a unified System-One control plane for both memory construction and retrieval, including multi-relational organization, query routing, adaptive graph traversal, candidate scoring, and stopping.
- Jev-Mem achieves the best overall accuracy while also delivering the highest efficiency among state-of-the-art baselines.

2 Background and Motivation

2.1 Agentic Memory

Figure 1 demonstrates the workflow of agentic memory. Let an agent maintain an evolving memory M_t . At interaction step t , a query q_t retrieves relevant evidence

$$E_t = R(q_t, M_t) \quad (1)$$

which is then provided to a language model for reasoning and response generation:

$$o_t = L(q_t, E_t) \quad (2)$$

The resulting interaction may subsequently update the memory:

$$M_{t+1} = U(M_t, q_t, o_t) \quad (3)$$

Agentic memory has evolved from simple retrieval over stored interaction histories toward increasingly active memory management. Modern systems may selectively organize observations, infer relationships among memories, consolidate information, route queries across different memory structures, and adapt retrieval based on the current query. As a result, memory is no longer only a passive store accessed by a retrieval function. It increasingly behaves as a dynamic subsystem that continuously makes decisions about how information should be organized and accessed.

This shift introduces an important systems question that has received comparatively less attention: *what computational mechanism should execute these memory-management decisions?* Both memory update U and retrieval R contain frequent semantic decisions. During memory construction, the system must determine how new information should be characterized and connected to existing knowledge. During retrieval, it must determine where to search, which candidates are useful, how much additional search is warranted, and when sufficient evidence has been collected.

External retrieval has long served as a mechanism for augmenting parametric language-model knowledge with non-parametric stores, through dense retrievers, retrieval-augmented pre-training, and retrieval-augmented generation (Karpukhin et al., 2020; Guu et al., 2020; Lewis et al., 2020; Borgeaud et al., 2022; Izacard et al., 2022). Agentic memory inherits much of this machinery, but differs in that its store is written by the agent’s own interaction history and must be maintained and re-organized over time rather than fixed in advance.

A closely related line of work studies adaptive control in retrieval-augmented generation. Rather than applying the same retrieval procedure to every query, active and adaptive methods dynamically determine whether retrieval is needed, when additional evidence should be acquired, or which retrieval strategy a given query warrants (Trivedi et al., 2023; Jiang et al., 2023; Asai et al., 2023; Jeong et al., 2024). These results motivate viewing retrieval as a controlled decision process rather

than a fixed top- k operation. Jev-Mem builds on this view, and extends the same principle from evidence acquisition to the broader memory lifecycle, including memory construction, query routing, budget allocation, graph traversal, and stopping.

Jev-Mem focuses on this memory-control layer. Rather than treating these decisions as incidental components embedded inside prompts or fixed heuristics, we make them an explicit part of the memory architecture.

2.2 The Opportunity for System-One Memory Control

A key observation is that many memory-control operations are *semantic but not generative*. During memory construction, the system may need to classify a memory or infer its relation to existing information; during retrieval, it may need to route a query, score candidates, or decide when to stop. These decisions typically produce bounded outputs such as labels, probabilities, or scores rather than free-form text.

Using an autoregressive LLM for such high-frequency decisions is therefore unnecessarily expensive: even simple judgments require token-by-token generation, formatting, and parsing. Because these operations repeatedly appear on the memory critical path, their overhead can accumulate quickly.

More broadly, cost-aware LLM inference systems have shown that different requests need not receive identical amounts of model computation. Cascading and routing approaches dynamically allocate requests across models of differing cost to improve the cost–quality tradeoff (Chen et al., 2024; Ong et al., 2025), and speculative decoding uses a small model to draft tokens that a larger model only verifies (Leviathan et al., 2023). Jev-Mem applies a related systems principle at a finer granularity: rather than routing only complete user requests between models, it separates frequent, bounded memory-control decisions from open-ended generative and deliberative reasoning.

This creates a natural opportunity for a System-One/System-Two design: use lightweight structured prediction for frequent memory-control decisions, while reserving System Two for complex reasoning and answer synthesis.

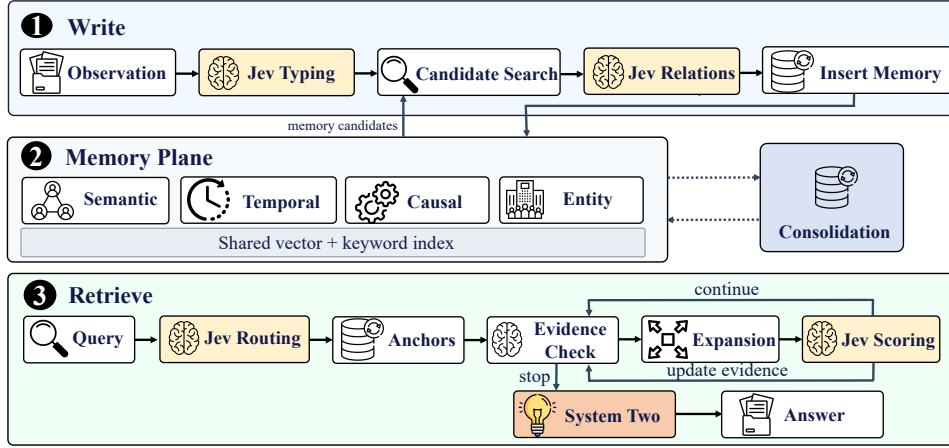


Figure 2: Overview of Jev-Mem. A lightweight System-One control plane manages memory construction and adaptive retrieval, while System Two is reserved for complex reasoning and answer synthesis.

3 Jev-Mem Algorithm Design

Jev-Mem redesigns agentic memory around a clear separation between fast memory control and deliberative reasoning. As illustrated in Figure 2, the architecture consists of three components: a System-One controller, a shared memory data plane, and a System-Two reasoning model. Rather than relying on an autoregressive LLM to manage every memory operation, the System-One controller makes lightweight, structured decisions throughout both writing and retrieval. The memory plane maintains canonical observations together with semantic, temporal, causal, and entity relations, while System Two is reserved for answer synthesis and other operations that require open-ended generation or deeper reasoning.

This architecture unifies memory construction and retrieval under the same control mechanism. On the write path, Jev-Mem preserves observations, assigns memory types, selects candidate memories, and determines how new observations should connect to the existing multi-relational memory. On the read path, it routes each query to the most relevant relational views, allocates retrieval effort, checks evidence sufficiency, expands the graph when necessary, and scores newly discovered candidates in an iterative feedback loop. By treating both workflows as coordinated System-One control processes, Jev-Mem replaces a collection of isolated heuristics and repeated LLM calls with a single, adaptive memory-management architecture.

Let an observation be

$$o_t = (x_t, \tau_t, \mu_t) \quad (4)$$

where x_t denotes its content, τ_t an optional times-

tamp, and μ_t its provenance. Jev-Mem maintains

$$\mathcal{M}_t = \left(V_t, \{E_t^g\}_{g \in \mathcal{G}}, I_t^{\text{vec}}, I_t^{\text{lex}} \right) \quad (5)$$

where

$$\mathcal{G} = \{\text{semantic, temporal, causal, entity}\} \quad (6)$$

All relational views share the same canonical memory nodes V_t ; multiple typed edges may connect the same pair of memories, while the vector and lexical indexes provide complementary entry points into the same memory space.

3.1 Typed System-One Memory Control

The central abstraction of Jev-Mem is a typed System-One controller $\mathcal{J}(S, Q)$, where S is structured state and Q is a batch of explicit decision questions. The controller produces either probabilities for independent propositions or a distribution over mutually exclusive alternatives. For example, it can estimate whether two memories are semantically related, whether a candidate is relevant to the current query, or whether the retrieved evidence is sufficient. When exactly one action is required, it selects among predefined alternatives such as temporal relations.

This interface is intentionally different from free-form LLM prompting. Each decision exposes a small, known output space, allowing Jev-Mem to represent memory control directly as probabilities rather than generating intermediate natural-language reasoning and subsequently parsing it. Moreover, decisions sharing the same state can be evaluated together in a single batched invocation.

The same interface governs both the write and read paths. On the write path, it controls memory

typing and relation construction. On the read path, it performs query routing, candidate evaluation, and stopping. This shared control plane is a key architectural property of Jev-Mem: memory is not constructed by one collection of heuristics and retrieved by another, but instead managed throughout its lifecycle through the same System-One decision abstraction.

3.2 System-One-Guided Memory Construction

For each valid observation, Jev-Mem creates one canonical memory node and selectively constructs relations around it. The current design preserves observations rather than making an irreversible learned store-or-discard decision at ingestion time. This prevents information that appears unimportant initially from being permanently lost before a future query reveals its relevance. Selectivity is instead introduced when memory structure is constructed and later when memory is retrieved.

The controller first predicts four overlapping memory characteristics:

$$\mathbf{t}(v) = (t_{\text{episodic}}, t_{\text{semantic}}, t_{\text{procedural}}, t_{\text{preference}}) \quad (7)$$

These scores annotate the node rather than assigning it to a single mutually exclusive category. The canonical node retains the original observation, provenance, timestamp, embedding, entities, and type scores.

A central challenge is determining how a new observation relates to an increasingly large memory. Comparing it against every existing node would make controller cost grow directly with memory size. Jev-Mem therefore separates *candidate discovery* from *relation judgment*. Deterministic retrieval first combines vector similarity, lexical overlap, shared entities, and temporal proximity to identify at most K_w candidates:

$$C(v) = \text{TopK}_{u \in V_t} s_{\text{cand}}(v, u) \quad (8)$$

The System-One controller then evaluates only these candidate pairs.

For every pair (v, u) , Jev estimates semantic relatedness, directional causal influence, same-episode membership, and, when necessary, entity equivalence. Whenever reliable structured information is already available, Jev-Mem avoids unnecessary learned inference. Timestamp ordering directly creates temporal relations, while exact shared identifiers directly create entity relations. When

temporal order is implicit, the controller instead chooses among before, after, during, contains, overlaps, same_time, and unknown.

An inferred edge of type g is inserted only when

$$P(g \mid v, u) \geq \theta_{\text{rel}} \quad (9)$$

Because relations are independent views over a common memory space, the same pair of nodes may simultaneously exhibit semantic, temporal, causal, and entity relationships. This differs from assigning each memory to a single graph or duplicating the same observation across several memory stores. The write path is

$$o_t \rightarrow \text{type} \rightarrow \text{candidates} \rightarrow \text{relations} \rightarrow \mathcal{M}_t \quad (10)$$

This design improves memory quality in two ways. First, preserving canonical observations avoids premature information loss. Second, selective relation construction suppresses unnecessary graph connectivity while retaining relations that can later support semantic, temporal, causal, or entity-based reasoning.

Periodic maintenance further examines a bounded neighborhood for redundancy, contradiction, obsolescence, and useful additional links. These decisions enrich the memory structure without removing the original evidence. If a higher-level textual abstraction is required, the control plane may explicitly escalate an approved merge or promotion to System Two; generation is therefore an optional consequence of a structured control decision rather than the default mechanism for memory maintenance.

3.3 Adaptive System-One Retrieval

Jev-Mem treats retrieval as a closed-loop control process rather than a single top- k search. Given a query q , the controller first predicts the relevance of each relational view,

$$\mathbf{p}(q) = \{p_g(q)\}_{g \in \mathcal{G}} \quad (11)$$

together with a multi-hop requirement $h(q)$ and a recency importance score $r(q)$. Because these probabilities are evaluated independently, a query may activate several graph views simultaneously instead of being assigned to one discrete retrieval intent.

A relation type is active when

$$p_g(q) \geq \theta_{\text{act}} \quad (12)$$

Given a total graph-expansion budget B , Jev-Mem distributes search effort according to the predicted graph needs:

$$w_g(q) = \frac{p_g(q)^\gamma}{\sum_{j \in \mathcal{A}(q)} p_j(q)^\gamma} \quad (13)$$

where $\mathcal{A}(q)$ denotes the active graphs and γ controls the concentration of the allocation. After assigning a feasible minimum budget m to active graphs, the remaining budget is distributed proportionally:

$$b_g = m + \text{LRound}_g [(B - m|\mathcal{A}(q)|)w_g(q)] \quad (14)$$

The multi-hop prediction further determines the allowed traversal depth,

$$D(q) = \min \{D_{\max}, \max(1, \lceil D_{\max} h(q) \rceil)\} \quad (15)$$

This probabilistic routing mechanism serves two purposes. It avoids spending equal retrieval effort on relations that are unlikely to help the current query, while still allowing multiple forms of evidence to be explored when a question requires them.

Anchor retrieval. Before graph traversal, Jev-Mem identifies high-quality entry points using both semantic and lexical retrieval. Vector and keyword rankings are fused through reciprocal-rank fusion:

$$s_{\text{RRF}}(v, q) = \sum_{L \in \{L_{\text{vec}}, L_{\text{lex}}\}: v \in L} \frac{1}{\kappa + \text{rank}_L(v)} \quad (16)$$

where $\kappa = 60$. The highest-ranked nodes initialize the visited set and search frontier. Hybrid anchors provide robust starting points before the controller begins more expensive graph reasoning.

Evidence-guided expansion. After each retrieval round d , Jev-Mem evaluates the current evidence set E_d . Instead of blindly continuing until a fixed graph depth or node count is reached, the controller estimates

$$s_d : \text{evidence sufficiency}, \quad (17)$$

$$u_d : \text{expected utility of further retrieval}, \quad (18)$$

$$m_d : \text{missing required evidence}, \quad (19)$$

$$c_d : \text{unresolved contradiction}. \quad (20)$$

Retrieval terminates with sufficient evidence when

$$s_d \geq \theta_{\text{suff}} \wedge m_d < \theta_{\text{cont}} \wedge c_d < \theta_{\text{cont}} \quad (21)$$

It may also terminate when

$$u_d < \theta_{\text{cont}} \quad (22)$$

indicating that additional traversal is unlikely to improve the evidence even if the current evidence is not fully sufficient.

This stopping mechanism is important for both efficiency and retrieval quality. Stopping too early risks missing necessary evidence, whereas uncontrolled expansion introduces irrelevant memories that can distract downstream reasoning. Jev-Mem therefore explicitly models both evidence completeness and the expected benefit of further search.

When additional evidence is needed, Jev-Mem expands neighboring nodes under the relation-specific budgets b_g and global limits on nodes, edges, depth, controller calls, and latency. Each candidate v is then evaluated by the System-One controller along four complementary dimensions: query relevance a_v , relation usefulness ℓ_v , information novelty n_v , and support for the current evidence c_v .

These System-One predictions are combined with deterministic retrieval signals. For a candidate reached through graph type g , the transition score is

$$s(v | q, E_d) = \frac{1}{\sum_{i=1}^5 \lambda_i} \left[\begin{array}{l} \lambda_1 z_v + \lambda_2 a_v + \lambda_3 p_g(q) \ell_v \\ + \lambda_4 n_v + \lambda_5 (\pi_e + c_v)/2 \end{array} \right] \quad (23)$$

where z_v denotes embedding similarity and π_e is the stored edge weight. If timestamps are available, the score is adjusted using the query-specific recency prediction:

$$\rho_v = \frac{1}{1 + \max(0, \tau_* - \tau_v)/\text{day}} \quad (24)$$

$$\tilde{s}(v) = \frac{s(v) + 0.1r(q)\rho_v}{1 + 0.1r(q)} \quad (25)$$

The top W candidates form the next beam and are added to the accumulated evidence. Retrieval therefore follows an iterative feedback loop:

$$\begin{array}{l} \text{route} \rightarrow \text{retrieve} \rightarrow \text{assess} \\ \rightarrow \text{expand} \rightarrow \text{reassess} \end{array} \quad (26)$$

This process differs fundamentally from fixed top- k retrieval or static graph traversal. The relational views being explored, the allocation of search effort, the candidates retained at each round, and the decision to continue all depend on the query and on the evidence already collected. In this sense,

retrieval itself becomes a System-One-controlled decision process.

Once retrieval terminates, the highest-scoring K memories are passed to the System-Two LLM:

$$y = \text{SystemTwo}(q, E) \quad (27)$$

System Two performs the final synthesis over the selected evidence but is not involved in normal graph routing, candidate expansion, or stopping decisions. Jev-Mem therefore concentrates expensive autoregressive reasoning where it is most valuable while using fast structured control to improve the quality and efficiency of the evidence supplied to it.

The control overhead remains explicitly bounded. A normal write requires one batched typing request and, when candidates exist, one batched relation request over at most K_w memory pairs. A query requires one routing request and, for each retrieval round, at most one evidence assessment and one batched candidate-scoring request. Independent limits on graph expansions, inspected edges, visited nodes, depth, controller invocations, and elapsed time prevent the control process itself from growing without bound.

4 Experiments

We conduct comprehensive experiments to evaluate Jev-Mem along two dimensions: reasoning effectiveness and system efficiency. We examine whether System-One-controlled memory construction and retrieval improve long-horizon question answering, and whether separating fast memory control from System-Two reasoning reduces the overhead of building and accessing persistent memory. Accordingly, we report answer accuracy together with memory construction time and per-query latency.

4.1 Experimental Setup

Datasets. We evaluate long-term conversational memory on two widely used benchmarks. **LoCoMo** (Maharana et al., 2024) contains ultra-long multi-session conversations and evaluates an agent’s ability to recover information requiring temporal, causal, and cross-session reasoning.

Baselines. We compare Jev-Mem with representative long-term memory approaches using the same backbone answer model whenever applicable.

- **Full Context:** provides the complete conversation history directly to the LLM without external memory.

- **A-MEM** (Xu et al., 2025): dynamically organizes and evolves memories based on agent experiences.
- **Nemori** (Nan et al., 2025): structures conversational memory through episodic segmentation and graph-based retrieval.
- **MemoryOS** (Kang et al., 2025a): organizes persistent memory using a hierarchical multi-level architecture.
- **MAGMA** (Jiang et al., 2026a): represents memories through multiple semantic, temporal, causal, and entity relations for structured retrieval.
- **Jev-Mem (ours):** uses a System-One controller to construct and adaptively retrieve from a multi-relational memory, while reserving the LLM for final reasoning and answer synthesis.

Metrics. We evaluate Jev-Mem from both accuracy and system-efficiency perspectives. For reasoning quality, we use the LLM-as-a-Judge score (Zheng et al., 2023), which measures whether the generated answer is correct with respect to the reference answer. For system efficiency, we report total memory construction time and average per-query latency. Memory construction time measures the end-to-end cost of building the persistent memory from the conversation history, while query latency measures the average time required to retrieve relevant evidence and produce an answer for each query. Together, these metrics capture both the effectiveness and runtime efficiency of the proposed memory architecture.

4.2 Overall Performance

Table 1 summarizes the performance of Jev-Mem and the baselines on LoCoMo. Jev-Mem achieves the highest overall LLM-as-a-Judge score of 0.777, compared with 0.700 for the strongest baseline, corresponding to an 11.0% relative improvement. The gains are also consistent across question types: Jev-Mem performs best in five of the six categories and matches the best result on temporal reasoning.

The improvement is especially clear on queries that require stronger retrieval and evidence selection. On Multi-Hop questions, Jev-Mem reaches 0.625, compared with 0.569 for the strongest baseline. On Open-Domain questions, it achieves 0.610, substantially higher than the best baseline score of 0.517. The largest gain appears in the Adversarial category, where Jev-Mem reaches 0.962, compared with 0.742 for the strongest baseline. Jev-Mem also

Table 1: Performance on the LoCoMo benchmark evaluated using the LLM-as-a-Judge metric. Higher scores indicate better performance. LLM model is based on gpt-4o-mini.

Method	Multi-Hop	Temporal	Open-Domain	Single-Hop	Adversarial	Overall
Full Context	0.468	0.562	0.486	0.630	0.205	0.481
A-MEM	0.495	0.474	0.385	0.653	0.616	0.580
MemoryOS	0.552	0.422	0.504	0.674	0.428	0.553
Nemori	0.569	0.649	0.485	0.764	0.325	0.590
MAGMA	0.528	0.650	0.517	0.776	0.742	0.700
Jev-Mem	0.623	0.637	0.618	0.802	0.962	0.777

Table 2: System efficiency comparison with total memory build time (in seconds) and average query latency (in seconds).

Method	Build Time (s)	Latency (s)
Full Context	N/A	1.74
A-MEM	3636	2.26
MemoryOS	3276	32.68
Nemori	1044	2.59
MAGMA	1404	1.47
Jev-Mem	158	0.93

achieves the highest Single-Hop score of 0.797 and matches the best Temporal score of 0.650.

These results indicate that Jev-Mem improves performance across a broad range of memory-intensive queries rather than specializing in a single reasoning pattern. The largest gains appear when the system must combine evidence across multiple memories or distinguish relevant information from plausible but misleading distractors. Jev-Mem addresses these cases through adaptive memory control: it estimates which relational views are useful for each query, allocates retrieval effort accordingly, evaluates candidate memories during traversal, and decides when the collected evidence is sufficient. As a result, retrieval is adapted to the needs of each query instead of following a fixed search strategy, allowing the reasoning model to operate on a more relevant and focused evidence set.

4.3 Efficiency of System-One Memory Control

Table 2 compares the system efficiency of Jev-Mem with the baselines in terms of total memory construction time and average query latency. Jev-Mem achieves substantial improvements in both stages. Its total memory build time is only 158 seconds, compared with 1,044 seconds for the fastest competing memory system, corresponding to an 84.9%

reduction, or a $6.6\times$ speedup. The gap is even larger compared with A-MEM and MemoryOS, whose memory construction requires more than 3,000 seconds.

Jev-Mem also achieves the lowest query latency, requiring only 0.93 seconds per query on average. This is 36.7% lower than the fastest memory-based baseline at 1.47 seconds, and 46.6% lower than directly processing the full context at 1.74 seconds. In contrast, MemoryOS requires 32.68 seconds per query, highlighting the substantial runtime overhead that memory management can introduce when complex processing remains on the retrieval path.

The efficiency gains reflect the separation of memory control from autoregressive reasoning in Jev-Mem. During memory construction, lightweight typed decisions are batched over a bounded candidate set rather than repeatedly invoking a general-purpose LLM for free-form memory processing. During retrieval, System-One control performs query routing, candidate evaluation, and evidence checking, while graph exploration is constrained by explicit search budgets. Adaptive stopping further avoids unnecessary traversal once sufficient evidence has been collected. Together, these mechanisms reduce the computational overhead of both constructing and accessing long-term memory.

Notably, the efficiency improvement does not come at the expense of reasoning quality. As shown in Table 1, Jev-Mem simultaneously achieves the highest overall accuracy on LoCoMo. These results demonstrate that separating fast memory control from System-Two reasoning can improve both the effectiveness and the efficiency of agentic memory.

5 Conclusion

We present **Jev-Mem**, a System-One/System-Two architecture for efficient agentic memory. Jev-Mem uses lightweight structured prediction for high-

frequency memory-control decisions and reserves System Two for complex reasoning and answer synthesis. On LoCoMo, Jev-Mem achieves the best overall accuracy and efficiency among the evaluated baselines, reaching an LLM-as-a-Judge score of 0.777, a memory construction time of 158 s, and an average query latency of 0.93 s. These results show that separating memory control from generative reasoning is a promising direction for building more effective and efficient long-horizon agents.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. [Self-rag: Learning to retrieve, generate, and critique through self-reflection](#). *Preprint*, arXiv:2310.11511.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. [Graph of thoughts: Solving elaborate problems with large language models](#). *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, and 9 others. 2022. [Improving language models by retrieving from trillions of tokens](#). *Preprint*, arXiv:2112.04426.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Lingjiao Chen, Matei Zaharia, and James Zou. 2024. [FrugalGPT: How to use large language models while reducing cost and improving performance](#). *Transactions on Machine Learning Research*. Featured Certification.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.
- Jonathan St. B. T. Evans. 2008. [Dual-processing accounts of reasoning, judgment, and social cognition](#). *Annual Review of Psychology*, 59(Volume 59, 2008):255–278.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, et al. 2026. Lightmem: Lightweight and efficient memory-augmented generation. In *International Conference on Learning Representations*, volume 2026, pages 98706–98729.
- Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. [LightRAG: Simple and fast retrieval-augmented generation](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10746–10761, Suzhou, China. Association for Computational Linguistics.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From rag to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. [Realm: Retrieval-augmented language model pre-training](#). *Preprint*, arXiv:2002.08909.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekesch, Fei Jia, and Boris Ginsburg. 2024. [RULER: What’s the real context size of your long-context language models?](#) In *First Conference on Language Modeling*.
- Yuanzhe Hu, Yu Wang, and Julian McAuley. 2026. [Evaluating memory in llm agents via incremental multi-turn interactions](#). *Preprint*, arXiv:2507.05257.
- Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2022. [Atlas: Few-shot learning with retrieval augmented language models](#). *Preprint*, arXiv:2208.03299.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. 2024. [Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity](#). In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, Mexico City, Mexico. Association for Computational Linguistics.
- Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. 2026a. Magma: A multi-graph based agentic memory architecture for ai agents. *arXiv preprint arXiv:2601.03236*.

- Dongming Jiang, Yi Li, Guanpeng Li, Qiannan Li, and Bingzhe Li. 2026b. Hage: Harnessing agentic memory via rl-driven weighted graph evolution. *arXiv preprint arXiv:2605.09942*.
- Dongming Jiang, Yi Li, Songtao Wei, Jinxin Yang, Ayushi Kishore, Alysa Zhao, Dingyi Kang, Xu Hu, Feng Chen, Qiannan Li, et al. 2026c. Anatomy of agentic memory: Taxonomy and empirical analysis of evaluation and system limitations. *arXiv preprint arXiv:2602.19320*.
- Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. [Active retrieval augmented generation](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, Singapore. Association for Computational Linguistics.
- Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems*, 37:59532–59569.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025a. Memory os of ai agent. *arXiv preprint arXiv:2506.06326*.
- Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025b. Memory os of ai agent. *arXiv preprint arXiv:2506.06326*.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. [Dense passage retrieval for open-domain question answering](#). In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.
- Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. 2024. A human-inspired reading agent with gist memory of very long contexts. *arXiv preprint arXiv:2402.09727*.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. [Fast inference from transformers via speculative decoding](#). *Preprint*, arXiv:2211.17192.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. 2026. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*.
- Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. 2025. Nemori: Self-organizing agent memory inspired by cognitive science. *arXiv preprint arXiv:2508.03341*.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. [RouteLLM: Learning to route LLMs from preference data](#). In *The Thirteenth International Conference on Learning Representations*.
- Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: Towards llms as operating systems.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22.
- Ofir Press, Noah A Smith, and Mike Lewis. 2021. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. [RAPTOR: Recursive abstractive processing for tree-organized retrieval](#). In *The Twelfth International Conference on Learning Representations*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. [Toolformer: Language models can teach themselves to use tools](#). *Preprint*, arXiv:2302.04761.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652.
- Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. 2025. Membench: Towards more comprehensive evaluation on the memory of llm-based agents. *arXiv preprint arXiv:2506.21605*.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. [Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions](#). *Preprint*, arXiv:2212.10509.

- TypeSafe AI. 2026. Typesafe ai. <https://typesafe.ai/>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*.
- Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. [Agent workflow memory](#).
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Jason Weston and Sainbayar Sukhbaatar. 2023. [System 2 attention \(is something you might need too\)](#). *Preprint*, arXiv:2311.11829.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*.
- Yilin Xiao, Zhehan Zhu, Yujing Zhang, Jin Chen, Zijin Hong, Luyao Zhuang, Qinggang Zhang, Shengyuan Chen, Xiaocao Ouyang, Lingfei Ren, et al. 2026. Zero-mem: Zero-token memory operations for llm agents. *arXiv preprint arXiv:2607.29377*.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. 2024. [SWE-agent: Agent-computer interfaces enable automated software engineering](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. [Tree of thoughts: Deliberate problem solving with large language models](#). *Preprint*, arXiv:2305.10601.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023b. [React: Synergizing reasoning and acting in language models](#). *Preprint*, arXiv:2210.03629.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. [Expel: Llm agents are experiential learners](#). *Preprint*, arXiv:2308.10144.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.
- Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. [Webarena: A realistic web environment for building autonomous agents](#). *Preprint*, arXiv:2307.13854.

A Appendix

A.1 Related Work

From memory storage to active memory management. Early work on agent memory established that long-term interaction requires more than retaining raw conversation history. Generative Agents (Park et al., 2023) maintain an experience stream and periodically synthesize higher-level reflections, while Reflexion (Shinn et al., 2023) stores linguistic feedback in episodic memory to improve future decisions. MemoryBank (Zhong et al., 2024) introduces long-term updating and forgetting, and MemGPT (Packer et al., 2023) manages information across different memory tiers through an operating-system-inspired virtual context. These systems shift memory from passive storage toward an active component that selects, transforms, and reuses past experience. A parallel line of work retains reusable behavior rather than conversational content: Voyager (Wang et al., 2023) accumulates a persistent skill library for an embodied agent, ExpeL (?) distills transferable insights from past trajectories, and Agent Workflow Memory (?) induces reusable workflows from prior action sequences. ReadAgent (Lee et al., 2024) takes a different route, compressing very long inputs into gist memories that are revisited on demand. Benchmarks such as LongMemEval (Wu et al., 2024), MemBench (Tan et al., 2025), and MemoryAgentBench (Hu et al., 2026) measure how well such systems retain and use information across long interactive horizons.

Structured and evolving agentic memory. Recent work increasingly organizes memory into richer and more adaptive structures. A-MEM (Xu et al., 2025) constructs interconnected memory notes and allows their representations to evolve as new information arrives. Mem0 (Chhikara et al., 2025) dynamically extracts and consolidates salient information and further explores graph-based memory for relational structure. MemoryOS (Kang et al., 2025b) organizes information across short-, mid-, and long-term memory tiers, while Nemori (Nan et al., 2025) structures interactions into coherent episodes and distills higher-level knowledge from them. Graph-based approaches further capture relationships that cannot be represented by semantic similarity alone. MAGMA (Jiang et al., 2026a), for example, organizes memories through semantic, temporal, causal, and entity relations. HAGE (Jiang et al.,

2026b) extends this direction with a weighted multi-relational graph in which query-conditioned routing and reinforcement learning jointly optimize relation features and traversal behavior, allowing retrieval paths to adapt to downstream reasoning objectives. Related structure also appears in retrieval systems that are not themselves agent-memory architectures: RAPTOR (Sarathi et al., 2024) builds recursive abstractive summary trees, GraphRAG (Edge et al., 2024) constructs entity graphs and community summaries to answer global queries, and LightRAG (Guo et al., 2025) pairs graph structure with incremental updates for efficiency. HippoRAG (Jimenez Gutierrez et al., 2024) and its continual-learning extension (Gutiérrez et al., 2025) make the link to memory more explicit, using graph-based association over a persistent non-parametric store. These systems share representational machinery with agentic memory, but target retrieval over a given corpus rather than the maintenance of an agent’s own accumulating experience, and they do not define the multi-relation semantics used here. Together, these systems show a broader shift from static memory stores toward structured, evolving, and query-adaptive memory representations.

Adaptive retrieval and memory access. Retrieval-augmented generation established external stores as a way to supplement parametric knowledge (Lewis et al., 2020; Guu et al., 2020; Izacard et al., 2022). Later work made the retrieval procedure itself adaptive: IRCot (Trivedi et al., 2023) interleaves retrieval with chain-of-thought steps, FLARE (Jiang et al., 2023) triggers retrieval during generation when the model becomes uncertain, Self-RAG (Asai et al., 2023) learns when to retrieve and how to critique the evidence it obtains, and Adaptive-RAG (Jeong et al., 2024) selects among retrieval strategies according to question complexity. These methods control evidence acquisition within a single generation episode: whether, when, and how much to retrieve from a corpus that they do not modify. Jev-Mem shares the view of retrieval as a learned, query-dependent decision, but situates that control in the agent-memory lifecycle, where the store is itself written and reorganized by the controller and where the decisions also include what to admit, how new information relates to existing memory, which relational view to traverse, and when to stop.

Efficiency of agent memory. As memory architectures become more sophisticated, the com-

putation required to construct, maintain, and retrieve memory has emerged as an equally important concern. SimpleMem (Liu et al., 2026) improves efficiency through semantic compression, asynchronous consolidation, and query-adaptive retrieval. LightMem (Fang et al., 2026) similarly separates lightweight online processing from more expensive memory consolidation and reduces the amount of computation required during interaction. More aggressively, Zero-Mem (Xiao et al., 2026) removes LLM generation from intermediate memory operations and retrieves directly over structured representations. These works demonstrate that memory quality and memory cost must be considered jointly rather than treating memory management as negligible compared with final answer generation. A complementary body of systems work reduces LLM inference cost directly, through model cascades and learned routers that dispatch each request to a model of appropriate capability (Chen et al., 2024; Ong et al., 2025). These concerns are distinct: lowering the cost of the memory pipeline, choosing which model answers a request, and controlling the fine-grained decisions taken inside the memory lifecycle. Jev-Mem targets the last of these.

System-One and System-Two computation. The terminology we adopt originates in dual-process accounts of human reasoning, which distinguish fast, automatic processing from slower and more effortful deliberation (Evans, 2008). We use this distinction as an analogy for how computation is allocated, not as a claim that the underlying mechanisms correspond. Analogous separations have been explored in LLM systems: System 2 Attention (Weston and Sukhbaatar, 2023) inserts an explicit reconsideration step before answering, while Tree of Thoughts (Yao et al., 2023a) and Graph of Thoughts (Besta et al., 2024) spend additional structured search at test time on problems that reward deliberation. These methods allocate deliberative computation within a single reasoning episode. Jev-Mem instead applies the distinction architecturally, to the memory subsystem itself.

Memory control as a systems abstraction. Jev-Mem builds on these developments but focuses on a different architectural question: *how should the decisions that govern memory be executed?* Existing work has primarily improved what is stored, how memories are organized, or how retrieval is performed. Jev-Mem instead treats memory control itself as a first-class systems plane. Inspired by the

System-One/System-Two distinction, it separates fast, structured memory decisions from deliberative language generation. A System-One controller governs both sides of the memory lifecycle—including memory typing, relation construction, query routing, retrieval-budget allocation, candidate scoring, evidence assessment, and adaptive stopping—over a shared structured memory plane, while System Two is reserved for synthesis and complex reasoning. This design provides a common control abstraction across memory construction and retrieval, targeting both reasoning quality and system efficiency rather than optimizing either stage in isolation.

B Jev Prompts for Memory Writing and Retrieval

This section describes the typed prompts used by Jev-Mem to construct and retrieve conversational memory. The quoted instructions and criteria are reproduced from the implementation, with candidate index 0 instantiated where applicable. The memories below are fictional and illustrate the request structure; no scores or retrieval outcomes are presented as measured Jev predictions.

B.1 Typed interface and request state

Jev-Mem submits a shared state object and a batch of typed questions through `TypeSafeClient.system_one`. Each Noul specifies a binary proposition using an instruction and explicit true and false criteria. Its returned value lies in $[0, 1]$. These model-reported values are not assumed to be calibrated probabilities. Multiple Nouns are evaluated as independent propositions rather than mutually exclusive labels; for example, an observation may be both episodic and semantic. Independence here concerns the question formulation, not statistical independence of the returned scores.

Question identifiers are bookkeeping keys and are not themselves model input. Consequently, every instruction names the state fields it uses. For candidate index i , relation and traversal keys are prefixed with `pair_i_` and `candidate_i_`, respectively. Questions in a batch do not consume one another’s answers. Choice is used when alternatives are mutually exclusive, as in the consolidation representation decision below; it returns a selected label and an option distribution.

Our running example contains memory `m1`, ob-

served on 14 May 2024 at 10:00: "Mira: My old bicycle broke.", and memory m2, observed on 16 May 2024 at 10:00: "Mira: I bought a new bicycle yesterday because my old one broke." Both have entity identifiers Mira and bicycle. The query is "When did Mira buy a new bicycle, and why?" Memory objects passed to Jev contain id, content, timestamp, and entities; the timestamp records when the statement was observed, rather than necessarily when the event occurred.

B.2 Memory writing

The active profile disables admission filtering (admission_enabled=false). Every valid nonempty observation therefore enters the write path; memory-type scores do not decide whether it is retained. The typing request contains state = {"observation": text} and four Noul: episodic, semantic, procedural, and preference. The following two templates illustrate event and preference typing.

Event typing episodic (Noul).

Instruction: Does observation describe a particular experience or event involving a participant?

true: A specific past, current or planned event, even if its exact time is unstated.

false: Only a general fact, procedure or preference with no particular event.

Preference typing preference (Noul).

Instruction: Does observation express a participant's preference, aversion or habitual choice?

true: An attributable like, dislike, preferred option or habitual choice.

false: An isolated action alone, another person's unattributed preference, or no preference evidence.

After typing, deterministic vector, keyword, entity and timestamp signals select at most 10 existing candidate memories. A relation request contains new_memory and a candidates list. For the running example, new_memory is m2 and candidates[0] is m1. The request batches semantic association and both causal directions; entity alias resolution is added only when exact entity identifiers do not already match.

Semantic relation semantic (Noul).

Instruction: Compare new_memory.content with candidates[0].content. Would a semantic link between these observations help retrieve a shared specific topic or fact?

true: A specific shared topic, fact or event makes the connection useful.

false: Only generic conversational vocabulary or no meaningful semantic connection.

Directed causal relation caused_by (Noul).

Instruction: Compare new_memory.content with candidates[0].content. Does the candidate event cause, enable or explain the event in new_memory.content?

true: The supplied accounts support this direction of causal influence.

false: Only similarity, chronology, a shared entity, or insufficient causal evidence.

A returned relation score of at least 0.60 creates the corresponding typed edge. For caused_by, the direction is candidate to new memory; causes tests the reverse direction. The causal criteria explicitly distinguish causal support from shared topics or mere chronology. Exact entity intersections are linked deterministically. In the example, the shared entity identifiers therefore make an alias-resolution prompt unnecessary.

B.3 Periodic consolidation

Every 20 successful Jev writes, the active profile evaluates candidate pairs for consolidation. Four Noul assess redundancy, contradiction, obsolescence and link usefulness; the following Choice selects a representation. This is a separate post-insertion decision, not an admission filter.

Representation choice representation (Choice).

Instruction: Compare new_memory.content with candidates[0].content. Which representation best fits the relationship between these two observations? Judge from the supplied accounts; do not assume answers to other questions.

keep_separate: Contradictory accounts, unique details that a combined representation would lose, or distinct facts/events without a supported general pattern.

merge: Compatible accounts of the same fact or event can be combined without losing unique details.

promote: Distinct repeated episodes explicitly support a stable general pattern suitable for semantic abstraction; prefer this over merge for repeated events.

uncertain: Insufficient evidence to choose a safe combined or separate representation.

Consolidation preserves raw observations. The normal periodic path records decisions and links; it does not automatically replace source memories. A caller-supplied System-Two summarizer can create

a new representation only when merge or promote is selected with probability at least 0.85 and the contradiction score is below 0.85. The selected-option probability, rather than a separate confidence summary, controls this threshold.

B.4 Retrieval routing and candidate scoring

The routing request contains only query. Six Nouns estimate semantic, temporal, causal and entity needs, together with `multi_hop_need` and `recency_importance`. The following templates illustrate time-sensitive and explanatory retrieval. A query may activate several graph types simultaneously.

Temporal routing `temporal` (Noun).

Instruction: Does answering query require event dates, durations, ordering or changes over time?

true: A time relation is needed to answer correctly.

false: Dates or ordering are incidental to the answer.

Causal routing `causal` (Noun).

Instruction: Does answering query require explaining a cause, motivation, enabling condition or effect?

true: Causal or explanatory evidence is needed.

false: Only factual association or chronology is requested.

The active profile uses a total graph-expansion budget of 80. Graphs with positive need scores of at least 0.10 receive a minimum allocation of one; the remaining budget is distributed in proportion to the active need scores, with largest-remainder rounding. The probability exponent is 1.0. Vector and keyword search provide anchors before graph expansion; the budget does not imply one provider request per edge.

A traversal request contains query, currently selected evidence, and proposed candidates. Each candidate includes its memory fields, graph type, relation properties, and source/target identifiers so that direction is explicit. For illustration, evidence can contain `m1` and the candidate can be `m2`, reached through a causal edge from `m1` to `m2`. This is a schematic traversal state, not a claim that this tiny example necessarily requires expansion in an actual run.

Candidate relevance `relevance` (Noun).

Instruction: Does `candidates[0].content` contain a fact needed to answer query?

true: Direct answer evidence or a necessary intermediate fact.

false: Only topic overlap or unrelated content.

Additional evidence `new_information` (Noun).

Instruction: Does `candidates[0].content` add an answer-relevant detail absent from evidence?

true: A distinct relevant detail or missing reasoning link.

false: Only duplicated evidence or irrelevant new details.

Two further Nouns assess `relation_usefulness` and `supports_current_evidence`. The implementation combines these four scores with query-to-memory cosine similarity, the need score of the traversed graph, and the stored edge probability. A recency adjustment uses `recency_importance`. Candidates are ranked with this combined score and the beam width is 10; a high relevance score alone is not an unconditional admission to the retrieved evidence.

B.5 Evidence-based stopping

The stopping request contains query, the currently selected top-k evidence, and retrieval depth. It evaluates `evidence_sufficient`, `continue_useful`, `missing_evidence`, and `contradiction`. The following templates distinguish answer support from the prospective usefulness of another retrieval round.

Answer sufficiency `evidence_sufficient` (Noun).

Instruction: Does evidence contain support for every factual part of an answer to query?

true: A grounded answer can be given from these memories without inventing missing facts.

false: Any required fact or reasoning link is unsupported; related topics alone are insufficient.

Value of continued retrieval `continue_useful` (Noun).

Instruction: Given query and evidence, is another retrieval round likely to fill a specific gap or resolve a conflict?

true: An identifiable missing fact or conflict could benefit from more memory retrieval.

false: No identifiable retrieval need remains or more memories are unlikely to help.

For temporal queries, evidence and candidate objects also include `timestamp_role` and `temporal_references`. These fields identify the timestamp as observation time and attach grounded expressions with their precision. Thus, sufficiency is judged with the same temporal grounding used to present the selected evidence to the answerer, rather than by treating every conversation date as an event date.

Evidence-based stopping requires `evidence_sufficient` to be at least 0.95 and both `missing_evidence` and `contradiction` to be below 0.15. Retrieval can also terminate when `continue_useful` is below 0.15. Independent limits bound depth (8), visited nodes (60), examined edges (2400), Jev attempts (16), and the retrieval time budget (15 seconds). These limits can terminate retrieval even if evidence is incomplete; the time budget is checked between operations and is not a strict preemption guarantee for local computation.

Jev controls evidence selection but does not generate the final free-form answer. The selected memories are passed to a separate System-Two language model. This separation makes the prompt examples above memory-control decisions, rather than answer-generation or evaluator prompts. Gold answers and benchmark evidence annotations are excluded from these Jev states.